

目录

- 1 实验目的**
- 2 实验要求及内容**
 - 2.1 读者-写者实验
 - 2.2 生产者-消费者实验
- 3 实验环境**
- 4 读者-写者问题实验**
 - 4.1 实验原理
 - 4.1.1 整体设计思想
 - 4.1.2 程序逻辑分析
 - 4.2 编译与运行方法
 - 4.2.1 编译命令
 - 4.2.2 运行方法
 - 4.2.3 测试说明
 - 4.2.3.1 测试样例格式
 - 4.2.3.2 测试样例
 - 4.3 实验结果分析
 - 4.3.1 测试用例1：基础场景
 - 4.3.1.1 理论分析
 - 4.3.1.2 实际运行结果
 - 4.3.2 测试用例2：写者密集场景
 - 4.3.2.1 理论分析
 - 4.3.2.2 实际运行结果
 - 4.3.3 测试用例3：读者密集场景
 - 4.3.3.1 理论分析
 - 4.3.3.2 实际运行结果
 - 4.3.4 测试用例4：交替读写场景
 - 4.3.4.1 理论分析
 - 4.3.4.2 实际运行结果
 - 4.3.5 测试用例5：写者饥饿
 - 4.3.5.1 理论分析
 - 4.3.5.2 实际运行结果
 - 4.4 实验总结
- 5 生产者-消费者问题实验**

5.1 程序设计说明

5.1.1 总体设计

5.1.2 数据结构与变量定义

5.1.3 核心算法设计

5.2 编译运行方法

5.2.1 编译命令

5.2.2 测试文件格式

5.2.3 运行命令

5.3 测试说明和实验结果分析

5.3.1 测试场景设计

5.3.2 测试用例1：单生产者单消费者

5.3.3 测试用例2：单生产者多消费者

5.3.4 测试用例3：多生产者单消费者

5.3.5 测试用例4：多生产者多消费者

5.3.6 测试总结

5.4 实验总结

6 实验总结与心得体会

7 附录A：提交材料清单

1 实验目的

本实验旨在让学生动手设计一个进程同步互斥的实验，更深刻的理解进程协作机制。openEuler 提供了互斥量(pthread_mutex_t)、信号量(sem_t)等同步对象和相应的系统调用，用于线程的互斥与同步。学生可通过对读者写者问题的调试或者生产者消费者问题的调试，了解 openEuler 中的同步、互斥机制。

2 实验要求及内容

2.1 读者-写者实验

利用 openEuler 信号量机制，实现读者写者问题的模拟。在 openEuler 环境下，创建一个包含 n 个线程的控制进程。用这 n 个线程 来表示 n 个读者或写者。每个线程按相应测试数据文件的要求，进行读写操作。请用信号量机制分别实现读者优先和写者优先的读者-写者问题。

- 读者-写者问题的读写操作限制：
 - 写-写互斥；
 - 读-写互斥；
 - 读-读允许；
- 读者优先的附加限制：如果一个读者申请进行读操作时已有另一读者正在进行读操作，则该读者可直接开始读操作。
- 写者优先的附加限制：如果一个读者申请进行读操作时已有另一写者在等待访问共享资源，则该读者必须等到没有写者处于等待状态后才能开始读操作。
- 运行结果显示要求：要求在每个线程创建、发出读写操作申请、开始读写操作和结束读写操作时分别显示一行提示信息，以确信所有处理都遵守相应的读写操作限制。

2.2 生产者-消费者实验

利用 openEuler 信号量机制，实现生产者消费者问题的模拟。在 openEuler 环境下，设计一个程序来解决有限缓冲问题，使用三个信号量：empty_sem（以记录有多少空位）、full_sem（以记录有多少满位）以及 mutex（二进制信号量或互斥信号量，以保护对缓冲插入与删除的操作）。对于本实验，empty_sem 与 full_sem 将采用标准计数信号量，而 mutex 将采用二进制信号量。生产者与消费者作为独立线程，在 empty_sem、full_sem、mutex 的同步前提下，对缓冲进行插入与删除。

3 实验环境

配置项	参数
本机操作系统	macOS Sequoia 15.7
CPU	Apple M3
虚拟机操作系统	openEuler 24.03 LTS
图形化桌面	UKUI
gcc	12.3.1

4 读者-写者问题实验

4.1 实验原理

4.1.1 整体设计思想

本实验中，读者-写者问题，我基于提供的程序进行。提供的程序使用**状态机模型**和**信号量机制**实现读者-写者问题的两种优先级策略，其核心思想包括：

- 使用状态变量 `state` 表示系统当前状态（空闲/读取/写入）
- 使用互斥锁 `mutex` 保护共享变量的访问
- 使用信号量 `Sig_read` 和 `Sig_wrt` 控制线程的阻塞与唤醒
- 使用计数器 `reader_count` 和 `writer_count` 跟踪线程数量

两种策略的核心差异如下：

策略	读者进入条件	线程完成后的优先级
读者优先	只要有读者在读就可以加入	写者完成时优先唤醒读者
写者优先	有写者等待时读者必须等待	读者/写者完成时优先唤醒写者

4.1.2 程序逻辑分析

以下，我结合提供的代码对程序的实现逻辑进行分析：

对于**读者优先算法**：

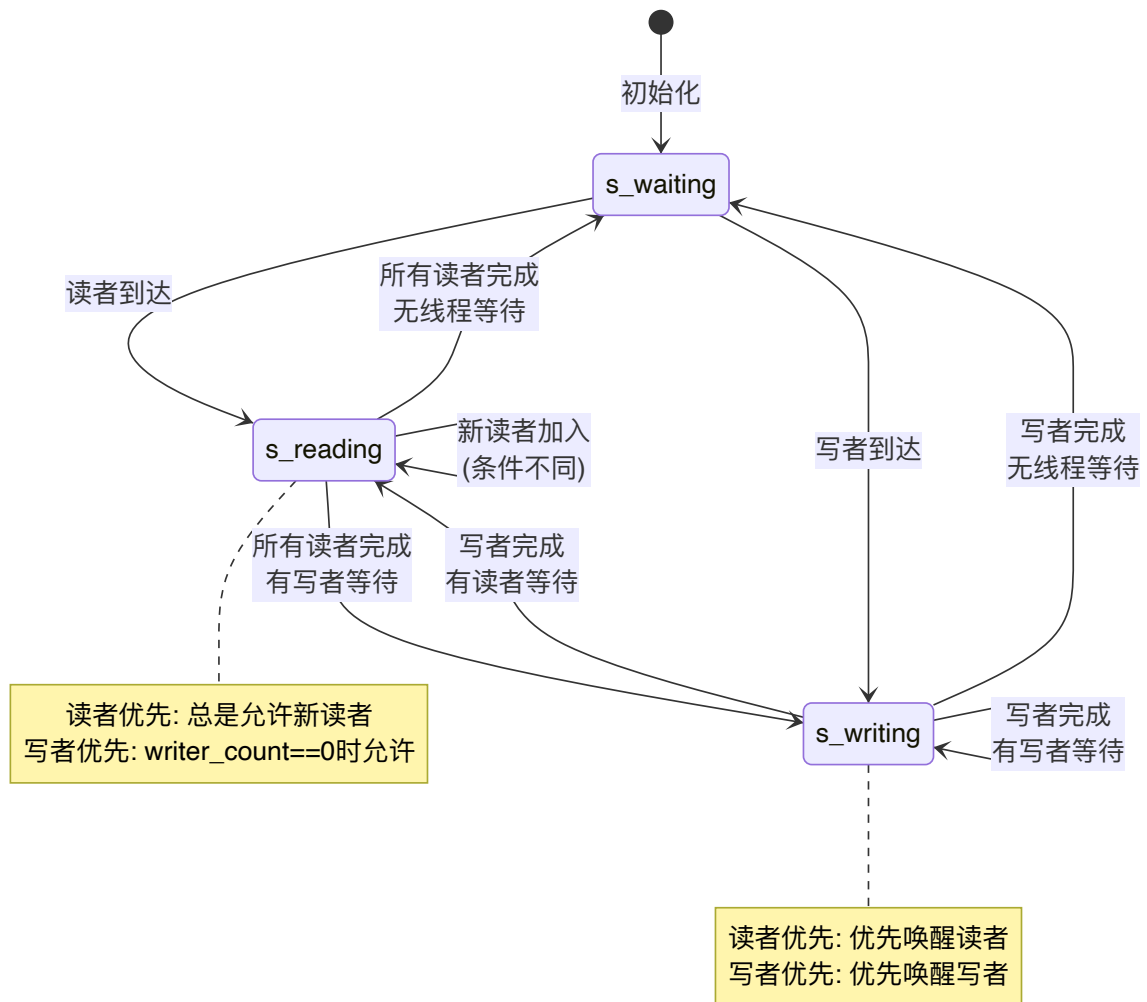
- 读者请求：若系统空闲或正在读取，立即允许进入
- 写者请求：仅在系统空闲时允许进入，否则等待

- 读者完成：最后一个读者离开时唤醒写者
- 写者完成：优先唤醒等待的读者

对于写者优先算法：

- 读者请求：若系统空闲或（正在读取且无写者等待），允许进入
- 写者请求：同读者优先
- 读者完成：最后一个读者离开时优先唤醒写者
- 写者完成：优先唤醒下一个写者，无写者时批量唤醒所有读者

可以画出程序的运行原理框图如下：



4.2 编译与运行方法

4.2.1 编译命令

```
1 # 编译读者优先程序
2 gcc readFirst.c -o readFirst.out -lpthread -std=c99
3
4 # 编译写者优先程序
5 gcc writeFirst.c -o writeFirst.out -lpthread -std=c99
```

其中：

- `-lpthread`：链接 pthread 线程库
- `-std=c99`：使用 C99 标准编译

4.2.2 运行方法

```
1 # 运行读者优先程序
2 ./readFirst.out < test
3
4 # 运行写者优先程序
5 ./writeFirst.out < test
```

4.2.3 测试说明

4.2.3.1 测试样例格式

1	线程ID	角色 (R/W)	延迟时间 (秒)	持续时间 (秒)
---	------	----------	----------	----------

4.2.3.2 测试样例

为了全面验证程序的正确性，我设计了 5 个测试用例，涵盖不同的读写场景：

test - 基础场景：

1	1 R 3 5
2	2 W 4 5
3	3 R 5 2
4	4 R 6 5
5	5 W 7 3

test2 - 写者密集场景： 用于验证多个写者竞争时的写-写互斥机制

1	1 W 2 3
2	2 W 3 4
3	3 R 4 2
4	4 W 5 3
5	5 W 6 2

test3 - 读者密集场景： 用于验证多个读者并发访问机制（读-读允许）。

1	1 R 2 3
2	2 R 3 4
3	3 R 4 2
4	4 R 5 5
5	5 W 6 3

test4 - 交替读写场景：用于验证读写频繁切换时的状态转换。

1	1 R 2 2
2	2 W 3 2
3	3 R 4 2
4	4 W 5 2
5	5 R 6 2
6	6 W 7 2

test5 - 写者饥饿：用于验证读者优先策略下的写者饥饿问题。

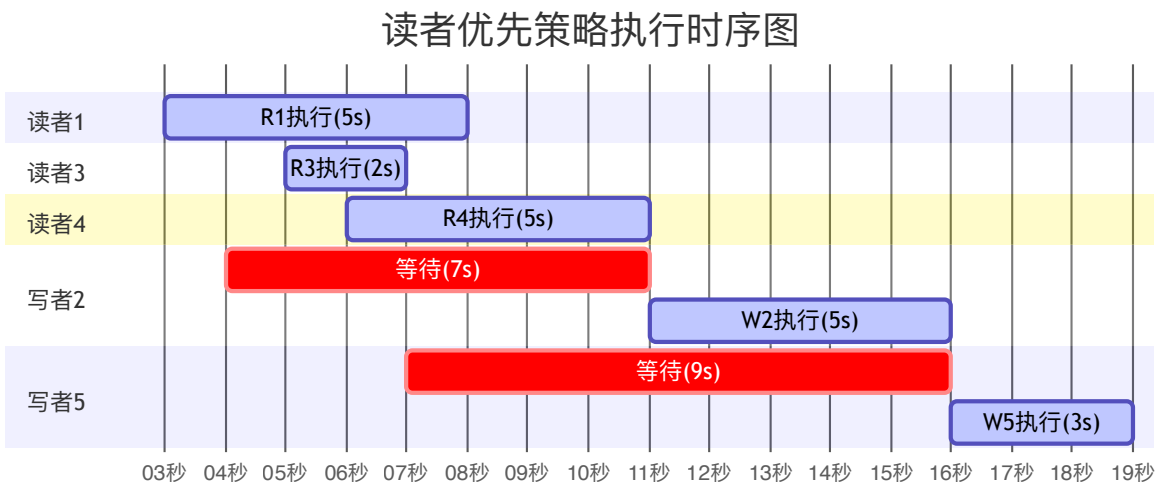
1	1 R 2 4
2	2 W 3 3
3	3 R 3 3
4	4 R 4 3
5	5 R 5 3
6	6 R 6 3
7	7 W 7 2

4.3 实验结果分析

4.3.1 测试用例1：基础场景

4.3.1.1 理论分析

采用读者优先策略时，理论执行时序图如下图所示：

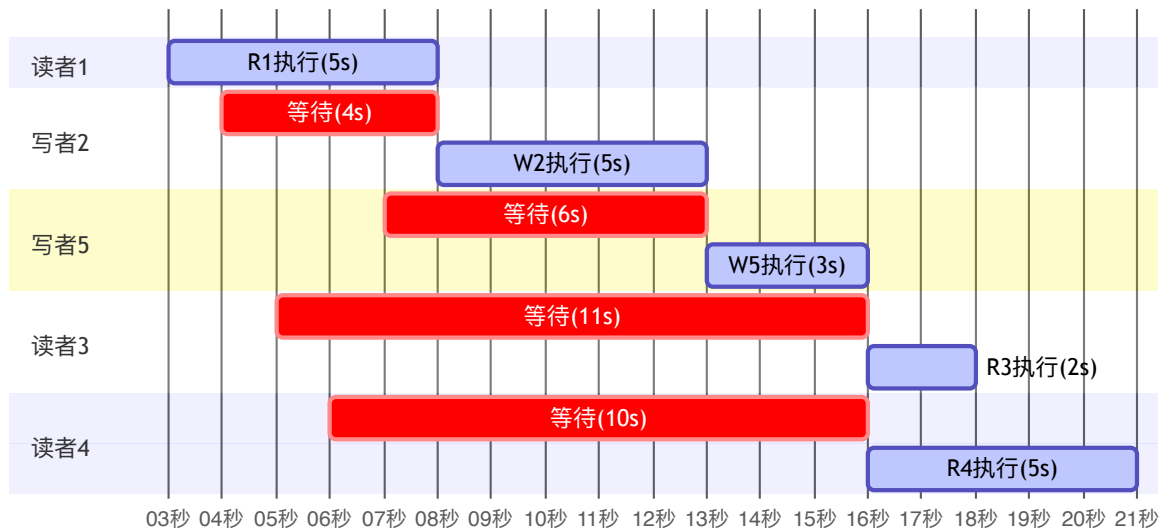


对于几个关键时间点的分析如下：

时间 (秒)	事件	系统状态	说明
t=0	创建所有线程	s_waiting	初始化完成
t=3	读者1申请读取	s_reading	系统空闲，立即进入
t=4	写者2申请写入	s_reading	有读者在读，进入等待队列
t=5	读者3申请读取	s_reading	state==s_reading，立即加入
t=6	读者4申请读取	s_reading	同样立即加入（体现读者优先）
t=7	读者3完成	s_reading	reader_count=2，写者2继续等待
t=7	写者5申请写入	s_reading	加入写者等待队列
t=8	读者1完成	s_reading	reader_count=1，写者仍需等待
t=11	读者4完成	s_writing	reader_count=0，唤醒写者2
t=16	写者2完成	s_writing	reader_count=0，writer_count=1，立即唤醒写者5
t=16	写者5开始写入	s_writing	无需等待，立即执行
t=19	写者5完成	s_waiting	所有线程完成

采用写者优先策略时，理论执行时序图如下图所示：

写者优先策略执行时序图



对于几个关键时间点的分析如下：

时间(秒)	事件	系统状态	说明
t=0	创建所有线程	s_waiting	初始化完成
t=3	读者1申请读取	s_reading	系统空闲，立即进入
t=4	写者2申请写入	s_reading	有读者在读，进入等待队列，writer_count=1
t=5	读者3申请读取	s_reading	writer_count>0，必须等待
t=6	读者4申请读取	s_reading	同样必须等待（体现写者优先）
t=7	写者5申请写入	s_reading	加入写者等待队列，writer_count=2
t=8	读者1完成	s_writing	writer_count>0，唤醒写者2
t=13	写者2完成	s_writing	writer_count=1，优先唤醒写者5
t=16	写者5完成	s_reading	writer_count=0，批量唤醒读者3和4
t=18	读者3完成	s_reading	-
t=21	读者4完成	s_waiting	所有线程完成

4.3.1.2 实际运行结果

读者优先输出：

```
[root@localhost src]# ./readFirst.out < test
[00秒]创建进程1
[00秒]创建进程2
[00秒]创建进程3
[00秒]创建进程4
[00秒]创建进程5
[03秒]读者进程1等待读取
[03秒]读者进程1开始读取
[04秒]写者进程2等待写入
[05秒]读者进程3等待读取
[05秒]读者进程3开始读取
[06秒]读者进程4等待读取
[06秒]读者进程4开始读取
[07秒]写者进程5等待写入
[07秒]读者进程3读取结束
[08秒]读者进程1读取结束
[11秒]读者进程4读取结束
[11秒]写者进程2开始写入
[16秒]写者进程2写入结束
[16秒]写者进程5开始写入
[19秒]写者进程5写入结束
```

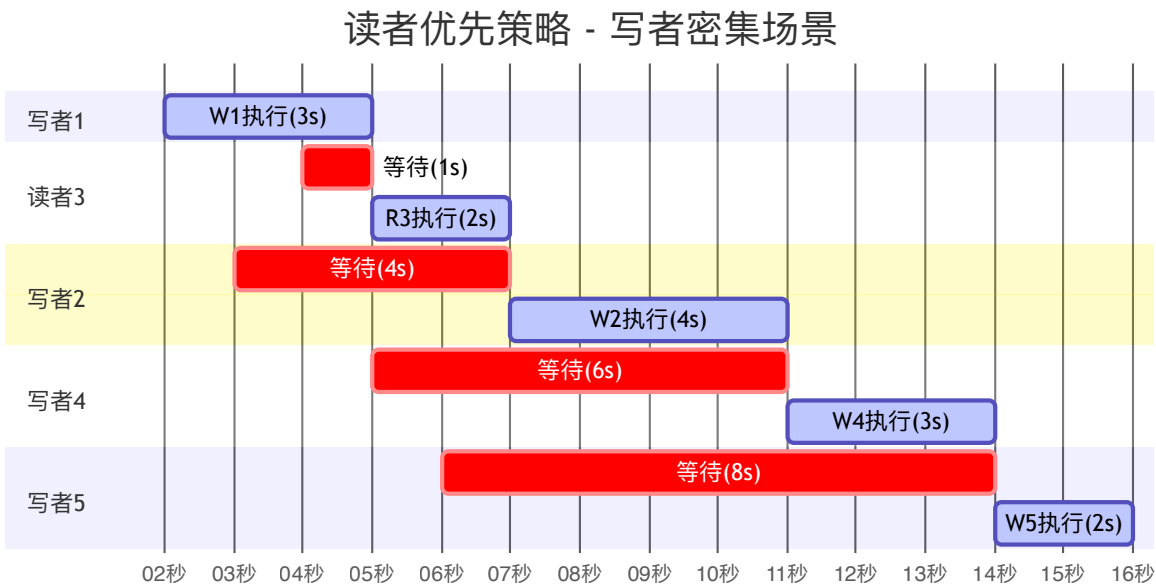
写者优先输出：

容易验证，实际运行结果与 Gantt 图中的理论分析完全一致。

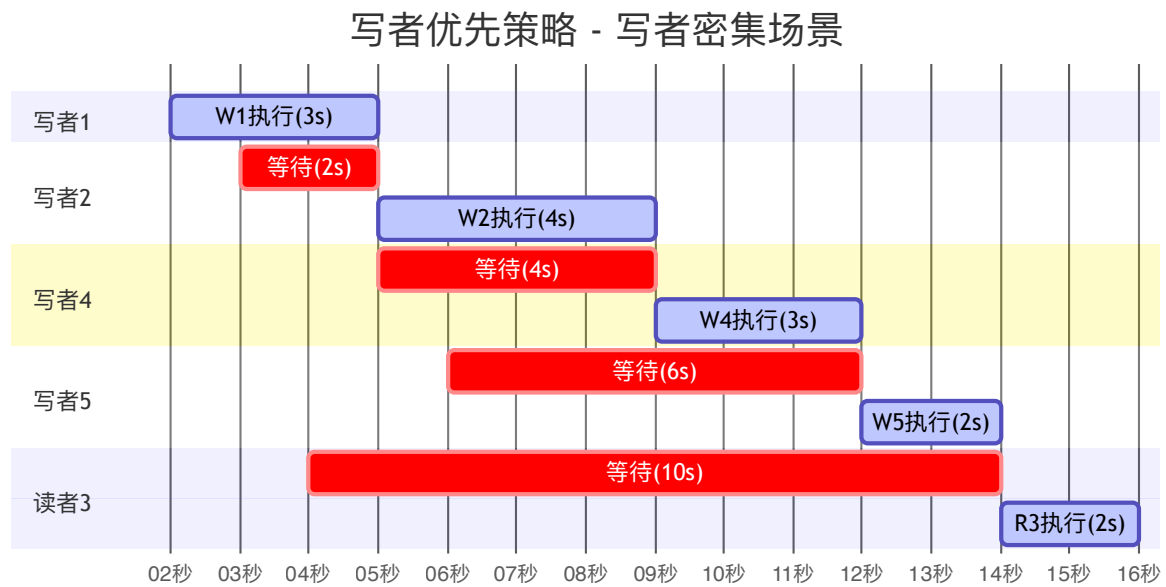
4.3.2 测试用例2：写者密集场景

4.3.2.1 理论分析

读者优先策略执行时序图：



写者优先策略执行时序图：



对于几个关键时间点的分析如下：

- $t=5$: 写者1完成后，优先唤醒写者2（体现写者优先）
- 写者连续执行：1→2→4→5
- 读者3被所有写者阻塞至 $t=14$

4.3.2.2 实际运行结果

读者优先输出：

```
[root@localhost src]# ./readFirst.out < test2
[00秒]创建进程1
[00秒]创建进程2
[00秒]创建进程3
[00秒]创建进程4
[00秒]创建进程5
[02秒]写者进程1等待写入
[02秒]写者进程1开始写入
[03秒]写者进程2等待写入
[04秒]读者进程3等待读取
[05秒]写者进程4等待写入
[05秒]写者进程1写入结束
[05秒]读者进程3开始读取
[06秒]写者进程5等待写入
[07秒]读者进程3读取结束
[07秒]写者进程2开始写入
[11秒]写者进程2写入结束
[11秒]写者进程4开始写入
[14秒]写者进程4写入结束
[14秒]写者进程5开始写入
[16秒]写者进程5写入结束
```

写者优先输出：

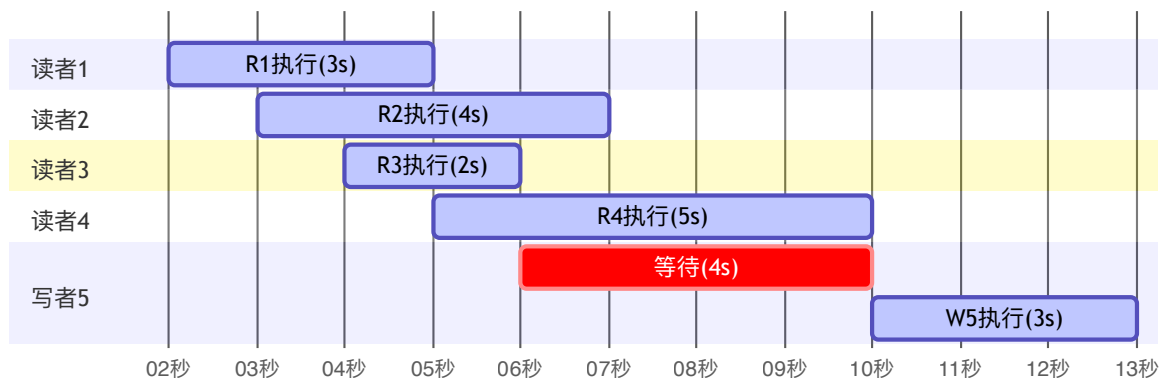
```
[root@localhost src]# ./writeFirst.out < test2
[00秒]创建进程1
[00秒]创建进程2
[00秒]创建进程3
[00秒]创建进程4
[00秒]创建进程5
[02秒]写者进程1等待写入
[02秒]写者进程1开始写入
[03秒]写者进程2等待写入
[04秒]读者进程3等待读取
[05秒]写者进程1写入结束
[05秒]写者进程4等待写入
[05秒]写者进程4开始写入
[06秒]写者进程5等待写入
[08秒]写者进程4写入结束
[08秒]写者进程2开始写入
[12秒]写者进程2写入结束
[12秒]写者进程5开始写入
[14秒]写者进程5写入结束
[14秒]读者进程3开始读取
[16秒]读者进程3读取结束
```

容易验证，实际运行结果与 Gantt 图中的理论分析完全一致。

4.3.3 测试用例3：读者密集场景

4.3.3.1 理论分析

读者优先策略 - 读者密集场景



对于几个关键时间点的分析如下：

- 4个读者并发执行（峰值reader_count=4）
- 写者5在t=6申请，等待至t=10（所有读者完成）

4.3.3.2 实际运行结果

读者优先输出：

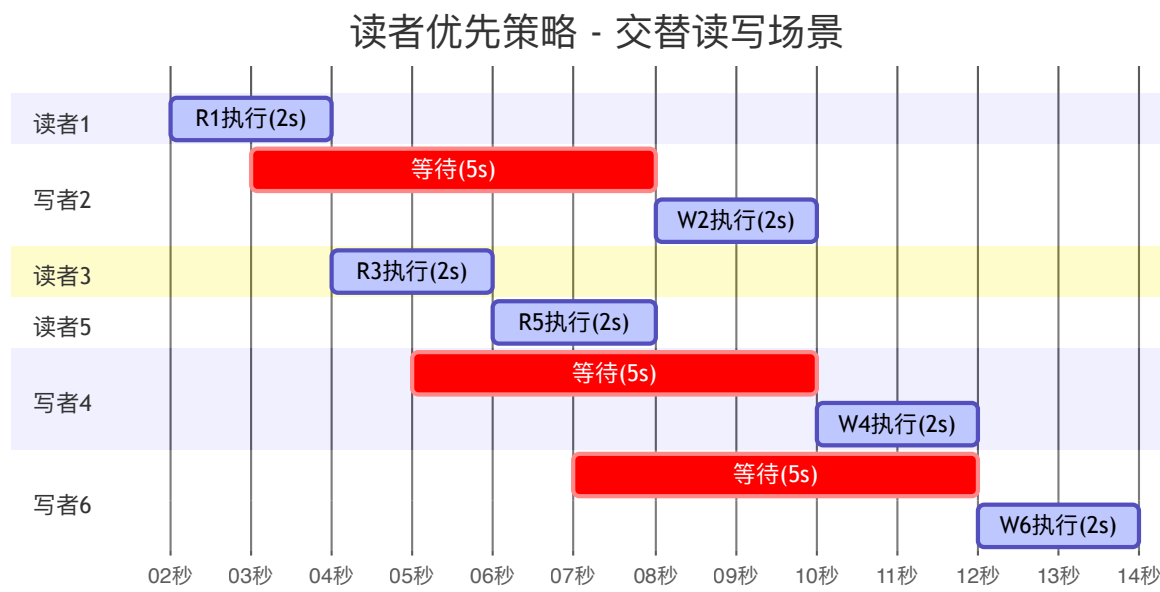
```
[root@localhost src]# ./readFirst.out < test3
[00秒]创建进程1
[00秒]创建进程2
[00秒]创建进程3
[00秒]创建进程4
[00秒]创建进程5
[02秒]读者进程1等待读取
[02秒]读者进程1开始读取
[03秒]读者进程2等待读取
[03秒]读者进程2开始读取
[04秒]读者进程3等待读取
[04秒]读者进程3开始读取
[05秒]读者进程1读取结束
[05秒]读者进程4等待读取
[05秒]读者进程4开始读取
[06秒]写者进程5等待写入
[06秒]读者进程3读取结束
[07秒]读者进程2读取结束
[10秒]读者进程4读取结束
[10秒]写者进程5开始写入
[13秒]写者进程5写入结束
```

容易验证，实际运行结果与 Gantt 图中的理论分析完全一致。

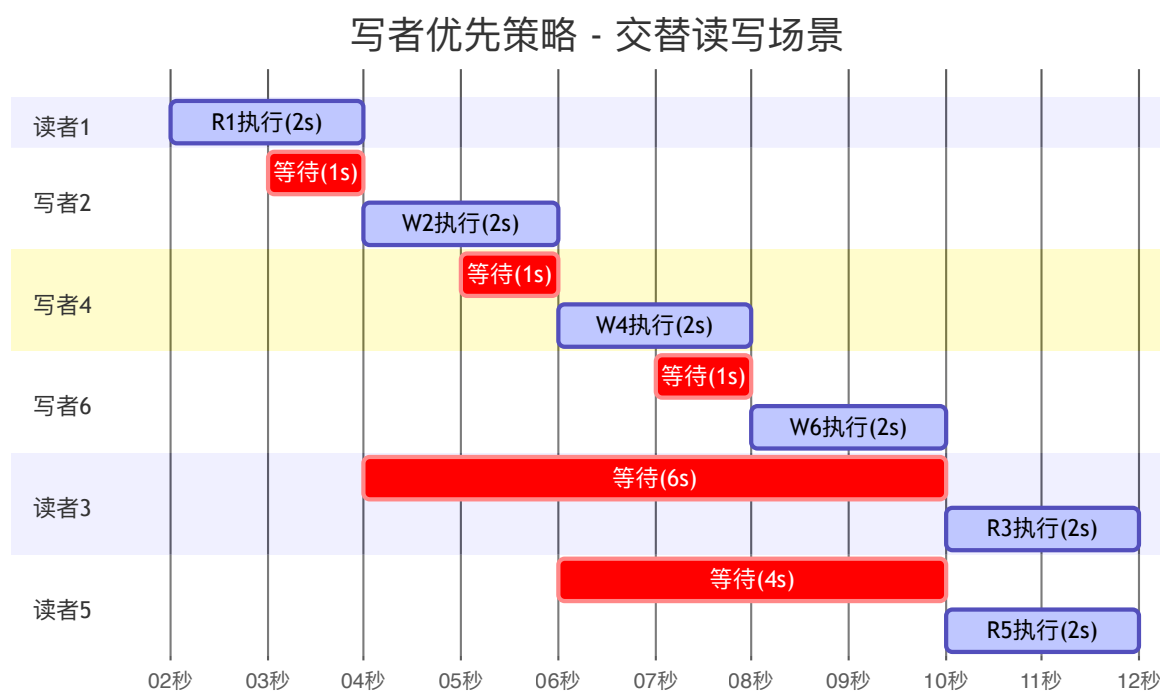
4.3.4 测试用例4：交替读写场景

4.3.4.1 理论分析

读者优先策略执行时序图：



写者优先策略执行时序图：



对于几个关键时间点的分析如下：

- 写者连续执行：2→4→6（体现写者优先）

- 读者3、5被阻塞至 t=10 后并发执行

4.3.4.2 实际运行结果

读者优先输出：

```
[root@localhost src]# ./readFirst.out < test4
[00秒]创建进程1
[00秒]创建进程2
[00秒]创建进程3
[00秒]创建进程4
[00秒]创建进程5
[00秒]创建进程6
[02秒]读者进程1等待读取
[02秒]读者进程1开始读取
[03秒]写者进程2等待写入
[04秒]读者进程3等待读取
[04秒]读者进程3开始读取
[04秒]读者进程1读取结束
[05秒]写者进程4等待写入
[06秒]读者进程5等待读取
[06秒]读者进程5开始读取
[06秒]读者进程3读取结束
[07秒]写者进程6等待写入
[08秒]读者进程5读取结束
[08秒]写者进程2开始写入
[10秒]写者进程2写入结束
[10秒]写者进程4开始写入
[12秒]写者进程4写入结束
[12秒]写者进程6开始写入
[14秒]写者进程6写入结束
```

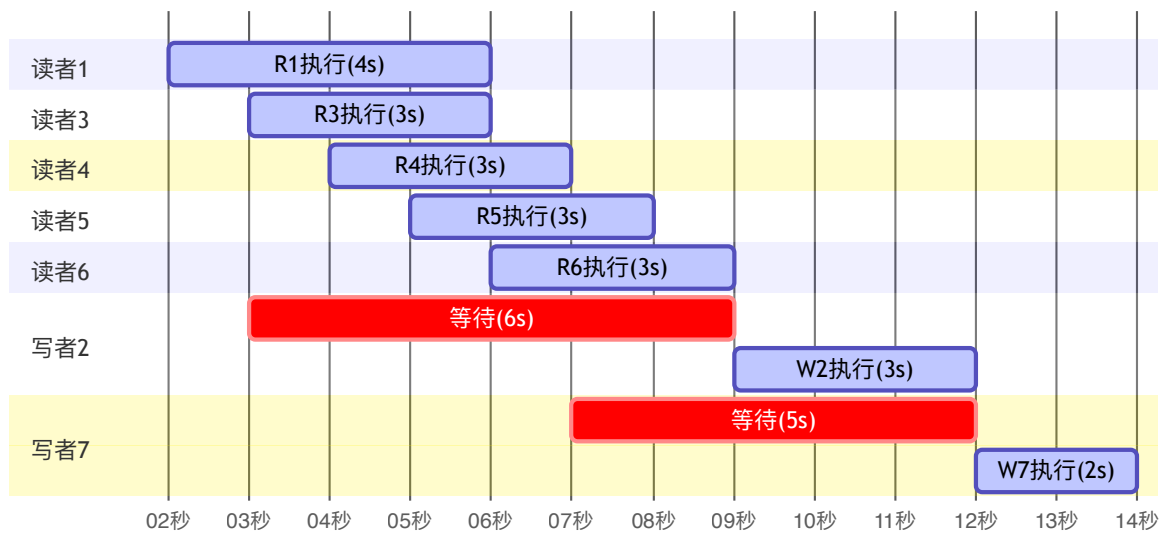
写者优先输出：

容易验证，实际运行结果与 Gantt 图中的理论分析完全一致。

4.3.5 测试用例5：写者饥饿

4.3.5.1 理论分析

读者优先策略 - 写者饥饿

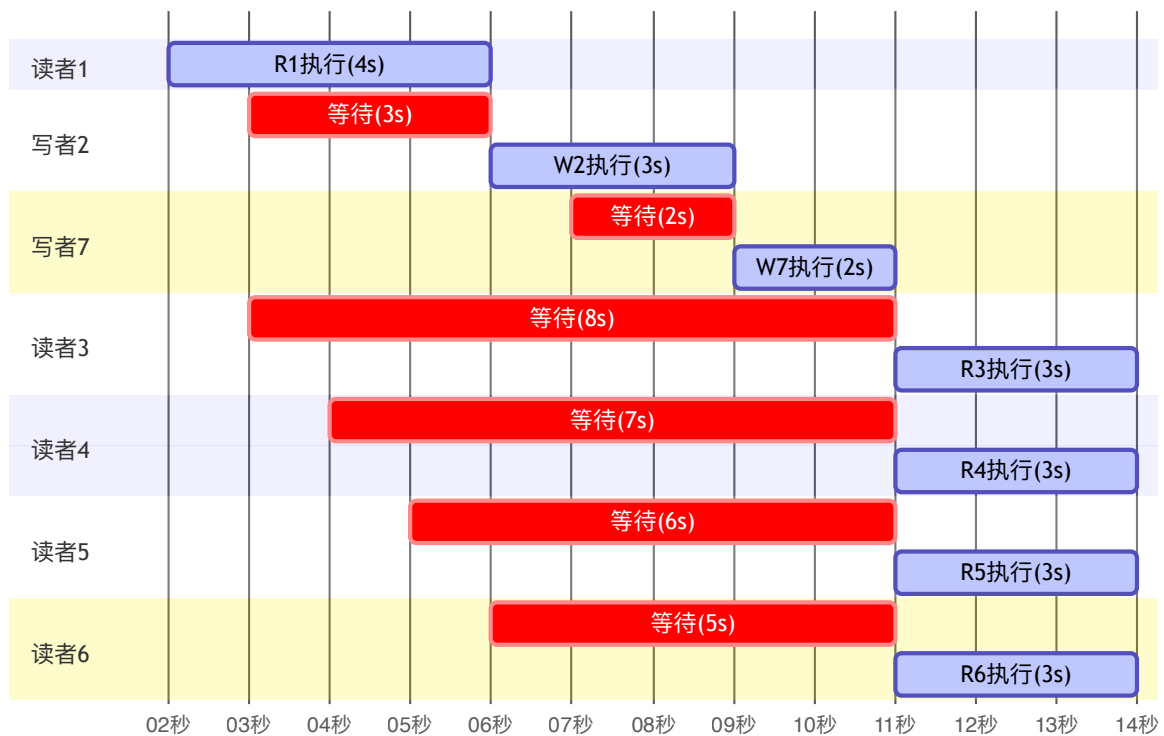


对于几个关键时间点的分析如下：

- 写者2在t=3申请，但被持续到达的读者阻塞至t=9
- 读者峰值并发：4个（t=5时刻）

写者优先策略执行时序图：

写者优先策略 - 避免写者饥饿



对于几个关键时间点的分析如下：

- 写者2、7连续执行，不被后续读者干扰
- 所有读者被阻塞至 $t=11$ ，然后并发执行

4.3.5.2 实际运行结果

读者优先输出：

```
[root@localhost src]# ./readFirst.out < test5
[00秒]创建进程1
[00秒]创建进程2
[00秒]创建进程3
[00秒]创建进程4
[00秒]创建进程5
[00秒]创建进程6
[00秒]创建进程7
[02秒]读者进程1等待读取
[02秒]读者进程1开始读取
[03秒]读者进程3等待读取
[03秒]读者进程3开始读取
[03秒]写者进程2等待写入
[04秒]读者进程4等待读取
[04秒]读者进程4开始读取
[05秒]读者进程5等待读取
[05秒]读者进程5开始读取
[06秒]读者进程1读取结束
[06秒]读者进程6等待读取
[06秒]读者进程6开始读取
[06秒]读者进程3读取结束
[07秒]写者进程7等待写入
[07秒]读者进程4读取结束
[08秒]读者进程5读取结束
[09秒]读者进程6读取结束
[09秒]写者进程2开始写入
[12秒]写者进程2写入结束
[12秒]写者进程7开始写入
[14秒]写者进程7写入结束
```

写者优先输出：

容易验证，实际运行结果与 Gantt 图中的理论分析完全一致。

读者优先场景下，读者3在t=3、读者4在t=4、读者5在t=5、读者6在t=6连续加入，持续阻塞写者2至t=9。

- 读者无需等待，立即开始读取
- 写者饥饿明显，写者2等待6秒，写者7等待5秒

而**写者优先**场景下，写者2、7连续执行，所有后续读者（3-6）被阻塞至t=11。

- 有效避免写者饥饿，写者2、7优先执行
- 但读者3-6等待8-11秒，响应时间长

4.4 实验总结

读者优先策略情景下读操作响应快，吞吐量高，适合读多写少场景，但容易造成写者饥饿。因此适合数据库查询系统、缓存系统等应用场景。

在采用**读者优先策略**下，如果需要解决写者饥饿问题，可通过设置读者最大并发数或写者等待时间阈值解决。

写者优先策略场景下能避免写者饥饿，保证数据及时更新。但读者可能长时间等待。更适用对数据一致性要求高的系统。

同样，在采用**写者优先策略**下，如果需要解决读者延迟问题，可通过设置写者队列上限或读者优先级动态调整解决。

5 生产者-消费者问题实验

5.1 程序设计说明

5.1.1 总体设计

生产者-消费者问题（Producer-Consumer Problem）是进程同步中的经典问题之一。我使用**有界缓冲区模型**和**纯信号量机制**实现生产者与消费者的协作。

5.1.2 数据结构与变量定义

```
1 // 缓冲区相关
2 int *buffer;           // 动态分配的循环缓冲区
3 int buffer_size;       // 缓冲区大小（从输入读取）
4 int total_products;    // 总生产数量（从输入读取）
5
6 // 索引指针
7 int produce_index = 0; // 下一个生产位置（循环队列）
8 int consume_index = 0; // 下一个消费位置（循环队列）
9
10 // 计数器
11 int produced = 0;      // 已生产数量（全局计数）
12 int consumed = 0;      // 已消费数量（全局计数）
13
14 // 信号量机制
15 sem_t mutex;           // 互斥信号量（初始值=1）
16 sem_t full_sem;        // 产品数量信号量（初始值=0）
17 sem_t empty_sem;       // 空位数量信号量（初始值=buffer_size）
18
19 // 时间
20 struct timeval start_time; // 程序启动时间基准、
```

5.1.3 核心算法设计

生产者算法：

```
1 void* producer(void* arg) {
2     initialize_random_seed();
3     sleep(info->delay);           // 延时启动
```

```

3
5     for (int i = 0; i < info->count; i++) {
6         // 1. 模拟随机生产耗时 (500-2000ms)
7         int produce_time = random_range(500, 2000);
8         usleep(produce_time * 1000);
9
10        // 2. 等待空位
11        sem_wait(&empty_sem);
12
13        // 3. 获取互斥锁
14        sem_wait(&mutex);
15
16        // 4. 生产产品 (临界区)
17        int product = ++produced;
18        buffer[produce_index] = product;
19        produce_index = (produce_index + 1) % buffer_size;
20
21        // 5. 释放互斥锁
22        sem_post(&mutex);
23
24        // 6. 通知消费者有新产品 (v操作)
25        sem_post(&full_sem);
26    }
27    return NULL;
28 }

```

消费者算法:

```

1 void* consumer(void* arg) {
2     initialize_random_seed();
3     sleep(info->delay);           // 延时启动
4
5     for (int i = 0; i < info->count; i++) {
6         // 1. 等待产品
7         sem_wait(&full_sem);
8
9         // 2. 获取互斥锁
10        sem_wait(&mutex);
11
12        // 3. 消费产品 (临界区)
13        int product = buffer[consume_index];
14        consumed++;
15        consume_index = (consume_index + 1) % buffer_size;
16
17        // 4. 释放互斥锁
18        sem_post(&mutex);
19
20        // 5. 通知生产者有空位
21        sem_post(&empty_sem);
22
23        // 6. 模拟随机消费耗时 (500-2000ms)
24        int consume_time = random_range(500, 2000);
25        usleep(consume_time * 1000);
26    }

```

```
27 |     return NULL;
28 | }
```

通过三个信号量：

- `mutex`：二元信号量（初始值1），实现对缓冲区的互斥访问
- `empty_sem`：计数信号量（初始值=buffer_size），记录空位数量
- `full_sem`：计数信号量（初始值0），记录产品数量

以实现进程间对临界区的访问控制，同时严格遵循"先资源信号量，后互斥锁"的顺序，避免死锁发生。

5.2 编译运行方法

5.2.1 编译命令

```
1 | gcc -pthread -std=c99 -Wall producer-consumer.c -o producer-consumer.out
```

5.2.2 测试文件格式

测试文件第一行为配置参数，后续每行描述一个线程：

```
1 | buffer_size total_products
2 | role delay count
3 | role delay count
4 | ...
```

参数说明：

- `buffer_size`：缓冲区大小
- `total_products`：总生产数量
- `role`：线程角色，P 表示生产者，C 表示消费者
- `delay`：线程启动延时（秒）
- `count`：生产/消费数量

5.2.3 运行命令

```
1 | ./producer-consumer.out < ../../test/producer-consumer/test1
```

5.3 测试说明和实验结果分析

5.3.1 测试场景设计

根据实验指导要求，我设计了四种典型场景：

测试用例	场景类型	缓冲区大小	总产品数	生产者数	消费者数	验证重点
test1	单生产者单消费者	3	10	1	1	基本互斥与同步
test2	单生产者多消费者	5	15	1	3	消费者竞争
test3	多生产者单消费者	5	15	3	1	生产者竞争
test4	多生产者多消费者	4	20	3	4	复杂并发

5.3.2 测试用例1：单生产者单消费者

测试数据：

1	3 10
2	P 1 10
3	C 2 10

理论分析：

单生产者单消费者是最基础的场景。生产者和消费者按序执行：生产者延时1秒启动，每次随机耗时500-2000ms生产一个产品放入缓冲区；消费者延时2秒启动，每次从缓冲区取出一个产品后随机耗时500-2000ms消费。由于缓冲区大小为3，当生产者连续生产3个产品后，若消费者未及时消费，生产者将在 `sem_wait(&empty_sem)` 处阻塞。

实际运行结果：

```

[root@localhost producer-consumer]# make test1
=== 测试1: 单生产者单消费者 ===
./producer-consumer.out < ../../test/producer-consumer/test1
[0000ms]程序启动, 缓冲区大小=3, 总生产量=10
[1000ms]生产者线程1启动, 计划生产10个产品
[1989ms]生产者1生产产品1, 放入位置[0] (耗时988ms)
[2001ms]消费者线程2启动, 计划消费10个产品
[2001ms]消费者2消费产品1, 从位置[0]取出
[3014ms]生产者1生产产品2, 放入位置[1] (耗时1025ms)
[3850ms]消费者2消费产品2, 从位置[1]取出
[4329ms]生产者1生产产品3, 放入位置[2] (耗时1314ms)
[4723ms]消费者2消费产品3, 从位置[2]取出
[5164ms]生产者1生产产品4, 放入位置[0] (耗时835ms)
[6056ms]消费者2消费产品4, 从位置[0]取出
[6713ms]生产者1生产产品5, 放入位置[1] (耗时1547ms)
[7528ms]消费者2消费产品5, 从位置[1]取出
[8322ms]生产者1生产产品6, 放入位置[2] (耗时1609ms)
[9141ms]消费者2消费产品6, 从位置[2]取出
[10276ms]生产者1生产产品7, 放入位置[0] (耗时1954ms)
[10276ms]消费者2消费产品7, 从位置[0]取出
[11875ms]生产者1生产产品8, 放入位置[1] (耗时1597ms)
[11875ms]消费者2消费产品8, 从位置[1]取出
[13763ms]生产者1生产产品9, 放入位置[2] (耗时1888ms)
[13763ms]消费者2消费产品9, 从位置[2]取出
[15296ms]生产者1生产产品10, 放入位置[0] (耗时1534ms)
[15296ms]生产者线程1结束
[15296ms]消费者2消费产品10, 从位置[0]取出
[16185ms]消费者线程2结束
[16185ms]所有线程结束, 总生产=10, 总消费=10

```

从运行结果可以验证, 消费者仅在缓冲区中有数据时消费, 未出现消费空数据的情况。且多个线程并发时缓冲区操作未出现竞态条件。

5.3.3 测试用例2: 单生产者多消费者

测试数据:

1	S 15
2	P 1 15
3	C 2 5
4	C 3 5
5	C 4 5

理论分析:

此场景验证多个消费者竞争产品的情况。1个生产者(延时1秒启动)负责生产15个产品, 3个消费者(分别延时2、3、4秒启动)各消费5个产品。由于消费者数量多于生产者, 消费者之间会在 `sem_wait(&full_sem)` 处竞争产品, 先被唤醒的消费者获得产品。`mutex` 信号量确保同一时刻只有一个消费者访问缓冲区, 避免数据竞争。随机消费时间使得消费者的调度顺序更具不确定性。

实际运行结果:


```

[root@localhost producer-consumer]# make test2
=== 测试2: 单生产者多消费者 ===
./producer-consumer.out < ../../test/producer-consumer/test2
[0000ms]程序启动, 缓冲区大小=5, 总生产量=15
[1000ms]生产者线程1启动, 计划生产15个产品
[2000ms]消费者线程2启动, 计划消费5个产品
[2731ms]生产者1生产产品1, 放入位置[0] (耗时1730ms)
[2731ms]消费者2消费产品1, 从位置[0]取出
[3000ms]消费者线程3启动, 计划消费5个产品
[4000ms]消费者线程4启动, 计划消费5个产品
[4351ms]生产者1生产产品2, 放入位置[1] (耗时1621ms)
[4351ms]消费者3消费产品2, 从位置[1]取出
[6097ms]生产者1生产产品3, 放入位置[2] (耗时1746ms)
[6097ms]消费者4消费产品3, 从位置[2]取出
[6624ms]生产者1生产产品4, 放入位置[3] (耗时525ms)
[6624ms]消费者2消费产品4, 从位置[3]取出
[7888ms]生产者1生产产品5, 放入位置[4] (耗时1264ms)
[7888ms]消费者3消费产品5, 从位置[4]取出
[8745ms]生产者1生产产品6, 放入位置[0] (耗时857ms)
[8745ms]消费者4消费产品6, 从位置[0]取出
[10184ms]生产者1生产产品7, 放入位置[1] (耗时1439ms)
[10184ms]消费者2消费产品7, 从位置[1]取出
[10874ms]生产者1生产产品8, 放入位置[2] (耗时689ms)
[10874ms]消费者4消费产品8, 从位置[2]取出
[12613ms]生产者1生产产品9, 放入位置[3] (耗时1738ms)
[12613ms]消费者3消费产品9, 从位置[3]取出
[13475ms]生产者1生产产品10, 放入位置[4] (耗时863ms)
[13475ms]消费者2消费产品10, 从位置[4]取出
[15084ms]生产者1生产产品11, 放入位置[0] (耗时1608ms)
[15084ms]消费者4消费产品11, 从位置[0]取出
[16054ms]生产者1生产产品12, 放入位置[1] (耗时970ms)
[16054ms]消费者2消费产品12, 从位置[1]取出
[16727ms]生产者1生产产品13, 放入位置[2] (耗时671ms)
[16727ms]消费者3消费产品13, 从位置[2]取出
[17658ms]消费者线程2结束
[18657ms]生产者1生产产品14, 放入位置[3] (耗时1930ms)
[18658ms]消费者4消费产品14, 从位置[3]取出
[19169ms]生产者1生产产品15, 放入位置[4] (耗时512ms)
[19169ms]生产者线程1结束
[19169ms]消费者3消费产品15, 从位置[4]取出
[19342ms]消费者线程4结束
[19694ms]消费者线程3结束
[19694ms]所有线程结束, 总生产=15, 总消费=15

```

可以验证, 多个消费者按照实际的调度顺序正确访问并消费生产者已经生产的产品。

5.3.4 测试用例3: 多生产者单消费者

测试数据:

1	S 15
2	P 1 5
3	P 2 5
4	P 3 5
5	C 1 15

理论分析:

此场景验证多个生产者竞争缓冲区空位的情况。3个生产者（分别延时1、2、3秒启动）各生产5个产品，1个消费者（延时1秒启动）负责消费全部15个产品。多个生产者会在 `sem_wait(&empty_sem)` 处竞争空位，并在访问缓冲区时通过 `mutex` 信号量互斥。由于生产时间随机（500-2000ms），生产者的生产顺序具有不确定性，但通过 `produced++` 保证产品编号全局递增且唯一。

实际运行结果：

```
[root@localhost producer-consumer]# make test3
=== 测试3: 多生产者单消费者 ===
./producer-consumer.out < ../../test/producer-consumer/test3
[0000ms]程序启动，缓冲区大小=5，总生产量=15
[1000ms]生产者线程1启动，计划生产5个产品
[1000ms]消费者线程4启动，计划消费15个产品
[2000ms]生产者线程2启动，计划生产5个产品
[2740ms]生产者1生产产品1，放入位置[0]（耗时1739ms）
[2740ms]消费者4消费产品1，从位置[0]取出
[3000ms]生产者线程3启动，计划生产5个产品
[3379ms]生产者2生产产品2，放入位置[1]（耗时1379ms）
[3518ms]消费者4消费产品2，从位置[1]取出
[3929ms]生产者1生产产品3，放入位置[2]（耗时1188ms）
[4221ms]生产者2生产产品4，放入位置[3]（耗时841ms）
[4505ms]生产者1生产产品5，放入位置[4]（耗时576ms）
[4506ms]生产者3生产产品6，放入位置[0]（耗时1505ms）
[4915ms]消费者4消费产品3，从位置[2]取出
[5421ms]生产者3生产产品7，放入位置[1]（耗时915ms）
[5943ms]生产者2生产产品8，放入位置[2]（耗时1721ms）
[6247ms]消费者4消费产品4，从位置[3]取出
[6466ms]生产者1生产产品9，放入位置[3]（耗时1961ms）
[7277ms]消费者4消费产品5，从位置[4]取出
[7277ms]生产者3生产产品10，放入位置[4]（耗时1322ms）
[9016ms]消费者4消费产品6，从位置[0]取出
[9016ms]生产者2生产产品11，放入位置[0]（耗时916ms）
[10120ms]消费者4消费产品7，从位置[1]取出
[10121ms]生产者1生产产品12，放入位置[1]（耗时858ms）
[10121ms]生产者线程1结束
[10940ms]消费者4消费产品8，从位置[2]取出
[10940ms]生产者3生产产品13，放入位置[2]（耗时950ms）
[11783ms]消费者4消费产品9，从位置[3]取出
[11783ms]生产者2生产产品14，放入位置[3]（耗时934ms）
[11784ms]生产者线程2结束
[13348ms]消费者4消费产品10，从位置[4]取出
[13348ms]生产者3生产产品15，放入位置[4]（耗时1644ms）
[13348ms]生产者线程3结束
[15068ms]消费者4消费产品11，从位置[0]取出
[15849ms]消费者4消费产品12，从位置[1]取出
[16777ms]消费者4消费产品13，从位置[2]取出
[17744ms]消费者4消费产品14，从位置[3]取出
[19600ms]消费者4消费产品15，从位置[4]取出
[21074ms]消费者线程4结束
[21074ms]所有线程结束，总生产=15，总消费=15
```

多个生产者根据实际调度情况依次生产产品，产品编号正确，并且在缓冲区操作未出现竞态条件。

5.3.5 测试用例4：多生产者多消费者

测试数据：

1	4 20
2	P 1 7
3	P 2 7
4	P 3 6
5	C 1 5
6	C 2 5
7	C 3 5
8	C 4 5

理论分析：

此场景是最复杂的并发测试。3个生产者（延时1、2、3秒启动）分别生产7、7、6个产品（共20个），4个消费者（均延时1秒启动）各消费5个产品。多个生产者竞争空位（`empty_sem`），多个消费者竞争产品（`full_sem`），所有线程通过 `mutex` 信号量保证缓冲区访问的互斥性。

实际运行结果：

```

[root@localhost producer-consumer]# make test4
=== 测试4: 多生产者多消费者 ===
./producer-consumer.out < ../../test/producer-consumer/test4
[8000ms]程序启动, 缓冲区大小=4, 总生产量=20
[1000ms]生产者线程1启动, 计划生产7个产品
[1000ms]消费者线程4启动, 计划消费5个产品
[1801ms]生产者1生产产品1, 放入位置[0] (耗时800ms)
[1801ms]消费者4消费产品1, 从位置[0]取出
[2000ms]消费者线程5启动, 计划消费5个产品
[2000ms]生产者线程2启动, 计划生产7个产品
[3000ms]生产者线程3启动, 计划生产6个产品
[3000ms]消费者线程6启动, 计划消费5个产品
[3395ms]生产者1生产产品2, 放入位置[1] (耗时1594ms)
[3395ms]消费者5消费产品2, 从位置[1]取出
[3574ms]生产者2生产产品3, 放入位置[2] (耗时1573ms)
[3574ms]消费者6消费产品3, 从位置[2]取出
[4000ms]消费者线程7启动, 计划消费5个产品
[4485ms]生产者2生产产品4, 放入位置[3] (耗时911ms)
[4485ms]消费者4消费产品4, 从位置[3]取出
[4889ms]生产者3生产产品5, 放入位置[0] (耗时1888ms)
[4889ms]消费者7消费产品5, 从位置[0]取出
[5031ms]生产者1生产产品6, 放入位置[1] (耗时1636ms)
[5031ms]消费者5消费产品6, 从位置[1]取出
[5526ms]生产者2生产产品7, 放入位置[2] (耗时1040ms)
[5526ms]消费者6消费产品7, 从位置[2]取出
[6091ms]生产者1生产产品8, 放入位置[3] (耗时1060ms)
[6091ms]消费者7消费产品8, 从位置[3]取出
[6414ms]生产者2生产产品9, 放入位置[0] (耗时888ms)
[6414ms]消费者4消费产品9, 从位置[0]取出
[6765ms]生产者3生产产品10, 放入位置[1] (耗时1876ms)
[6765ms]消费者5消费产品10, 从位置[1]取出
[6925ms]生产者2生产产品11, 放入位置[2] (耗时510ms)
[6926ms]消费者6消费产品11, 从位置[2]取出
[7945ms]生产者1生产产品12, 放入位置[3] (耗时1852ms)
[7945ms]消费者4消费产品12, 从位置[3]取出
[8305ms]生产者2生产产品13, 放入位置[0] (耗时1380ms)
[8305ms]消费者7消费产品13, 从位置[0]取出
[8481ms]生产者3生产产品14, 放入位置[1] (耗时1717ms)
[8482ms]消费者5消费产品14, 从位置[1]取出
[9552ms]生产者1生产产品15, 放入位置[2] (耗时1608ms)
[9552ms]消费者6消费产品15, 从位置[2]取出
[9849ms]生产者3生产产品16, 放入位置[3] (耗时1366ms)
[9849ms]消费者4消费产品16, 从位置[3]取出
[10086ms]生产者1生产产品17, 放入位置[0] (耗时534ms)
[10086ms]生产者线程1结束
[10086ms]消费者5消费产品17, 从位置[0]取出
[10277ms]生产者2生产产品18, 放入位置[1] (耗时1972ms)
[10277ms]生产者线程2结束
[10277ms]消费者7消费产品18, 从位置[1]取出
[10988ms]消费者线程5结束
[11278ms]生产者3生产产品19, 放入位置[2] (耗时1430ms)
[11279ms]消费者6消费产品19, 从位置[2]取出
[11510ms]消费者线程4结束
[11960ms]生产者3生产产品20, 放入位置[3] (耗时680ms)
[11960ms]生产者线程3结束
[11960ms]消费者7消费产品20, 从位置[3]取出
[12591ms]消费者线程6结束
[13841ms]消费者线程7结束
[13841ms]所有线程结束, 总生产=20, 总消费=20

```

高并发情况下，生产者和消费者行为基本交替进行，程序运行正常，无死锁或阻塞，且生产者和消费者对缓冲区操作未出现竞态条件，且消费者未出现消费空数据的情况。

5.3.6 测试总结

通过以上测试，验证了我编写的程序满足：

1. **互斥性验证**：所有测试中，`mutex` 信号量确保同一时刻只有一个线程访问缓冲区，没有出现数据竞争

2. **同步性验证**：生产者在缓冲区满时阻塞于 `sem_wait(&empty_sem)`，消费者在缓冲区空时阻塞于 `sem_wait(&full_sem)`，同步机制正确
3. **正确性验证**：所有测试均满足 `总生产数量 = 总消费数量 = total_products`，无产品丢失或重复消费
4. **并发性验证**：多生产者/多消费者场景下，线程正确协作，随机时间（500-2000ms）使并发调度更贴近实际场景

5.4 实验总结

生产者-消费者问题的核心在于**资源的有限性和访问的互斥性**。本实验通过**纯信号量机制**实现了：

1. **互斥访问**：`mutex` 信号量（初始值1）确保缓冲区操作的原子性
2. **资源同步**：`empty_sem`（初始值=buffer_size）和 `full_sem`（初始值0）实现了生产者与消费者的协调
3. **死锁避免**：严格遵循"先等待资源信号量（empty/full），再获取互斥锁（mutex）"的顺序

实验结果表明，无论是单生产者/单消费者、单生产者/多消费者、多生产者/单消费者还是多生产者/多消费者场景，程序都能正确处理并发，保证数据一致性。生产者-消费者模型广泛应用于消息队列系统，如 RabbitMQ、Kafka 的消息缓冲机制等多种实际场景。

6 实验总结与心得体会

本实验通过实现读者-写者问题和生产者-消费者问题两个经典的进程同步案例，深入理解了 openEuler 系统中的同步与互斥机制。

在**读者-写者问题**中，读者优先策略适合读多写少的场景，能提高读操作的并发度和响应速度，但可能导致写者饥饿问题；而写者优先策略能有效避免写者饥饿，保证数据的及时更新，但会增加读者的等待时间。在实际应用中，应根据具体业务场景（读写比例、响应时间要求、数据一致性要求）选择合适的策略，或采用改进的公平策略（如读写公平锁）。

在**生产者-消费者问题**中，我们学习了如何使用信号量机制实现资源的有序访问。通过 `empty_sem`、`full_sem` 和 `mutex` 三个信号量的协作，成功解决了缓冲区有限的问题，避免了死锁和资源竞争。

同时，由于线程调度的不可预测性，测试和验证程序的正确性变得尤为重要，我通过设计多种测试场景，能够全面验证程序的正确性和健壮性。

通过本次实验，我深刻体会到：信号量是实现进程同步的强大工具：通过合理使用 P/V 操作，可以优雅地解决复杂的并发问题。同时，必须仔细设计信号量的等待顺序，以避免死锁。

综上所述，这次实验不仅让我掌握了读者-写者问题和生产者-消费者问题的解决方法，还让我对多线程编程有了更深层次的理解。我学会了如何合理设计线程间的通信机制，如何利用信号量保障数据一致性，同时也意识到了多线程编程中的细节处理与优化空间。我将在以后的学习和实际应用的开发过程中进一步探索进程同步。

7 附录A：提交材料清单

本实验提交材料包含以下文件：

1	.	
2	docs/	# 文档目录
3	report.pdf	# 实验报告 (PDF 版)
4	report.md	# 实验报告
5	src/	# 源代码目录
6	reader-and-writer/	# 读者-写者问题
7	readFirst.c	# 读者优先源码
8	writeFirst.c	# 写者优先源码
9	Makefile	# 编译脚本
10	producer-consumer/	# 生产者-消费者问题
11	producer-consumer.c	# 源码
12	Makefile	# 编译脚本
13	test/	# 测试数据目录
14	reader-and-writer/	# 读者-写者测试数据
15	test	# 测试1: 基础场景
16	test2	# 测试2: 多读者场景
17	test3	# 测试3: 读写交替
18	test4	# 测试4: 多写者场景
19	test5	# 测试5: 复杂并发
20	producer-consumer/	# 生产者-消费者测试数据
21	test1	# 测试1: 单生产者单消费者
22	test2	# 测试2: 单生产者多消费者
23	test3	# 测试3: 多生产者单消费者
24	test4	# 测试4: 多生产者多消费者